Rapport d'industrialisation : prédiction du churn télécom

Projet fil rouge MLOps, ESMT · Ndeye Khadidiatou Lo ·

Résumé

Ce projet transforme un modèle de prédiction du churn télécom en un service prêt pour la production. Le modèle, une régression logistique entraînée sur 2 000 clients, est exposé par une API FastAPI, couvert par 21 tests automatisés, empaqueté dans une image Docker et livré par un pipeline GitHub Actions. Une pile de supervision complète (Prometheus, Grafana, ELK) suit son comportement en continu, et des tests de charge JMeter ont mesuré ses limites : aucune erreur jusqu'à 1 000 utilisateurs simultanés, avec un débit plafonnant autour de 220 requêtes par seconde.

Au fil du travail, 31 erreurs ou manques de l'énoncé ont été identifiés et corrigés, dont 9 qui empêchaient purement et simplement le pipeline de fonctionner. Ce rapport retrace les choix techniques, les résultats obtenus et ce qu'il resterait à faire pour une mise en production réelle.

1. Introduction

Contexte

Dans les télécommunications, conquérir un nouveau client coûte bien plus cher que garder un client existant. Le churn, c'est à dire le départ d'un abonné vers la concurrence, pèse donc directement sur le chiffre d'affaires. Si l'on sait repérer à l'avance les clients à risque, les équipes de fidélisation peuvent concentrer leurs efforts là où ils comptent : une offre ciblée, un appel du centre de contact, un traitement prioritaire des réclamations.

Problématique

Un modèle qui tourne dans un notebook ne sert à personne en dehors de son auteur. Pour qu'un CRM, une interface de conseiller ou un outil de campagne puisse l'interroger, il faut le transformer en service : accessible par le réseau, testé, reproductible, surveillé et facile à mettre à jour. C'est tout l'enjeu du MLOps, et c'est la question à laquelle ce projet répond.

Objectifs

Le projet devait livrer une chaîne complète, de la donnée brute jusqu'au service supervisé :

1. entraîner et comparer plusieurs modèles, puis retenir le meilleur sur des critères mesurés ;
2. exposer ce modèle par une API documentée ;
3. garantir sa qualité par des tests unitaires, d'intégration, de bout en bout et de charge ;
4. le conteneuriser et automatiser sa livraison ;
5. le superviser en production, par les logs comme par les métriques.

Tout le code est versionné dans le dépôt GitHub Umm-abdallah/ml-churn-project, avec une branche `main` pour la version stable et une branche `develop` pour l'intégration.

2. Données et modèle

Le jeu de données

Le jeu de données compte 2 000 clients, décrits par six variables : âge, ancienneté en mois, revenu, nombre de réclamations, montant mensuel et nombre de retards de paiement. La cible `churn` vaut 1 quand le client est parti. Il est généré par le script `src/generate_data.py` avec une graine fixe, ce qui garantit que chacun obtient exactement les mêmes données.

Les données sont propres : aucune valeur manquante, aucun doublon. Le taux de churn atteint 43 % (861 départs sur 2 000), les deux classes sont donc assez équilibrées pour se passer de rééchantillonnage.

Ce que révèle l'analyse exploratoire

L'analyse, menée dans le notebook `notebooks/exploration.ipynb`, fait ressortir une variable dominante : l'ancienneté. Deux clients sur trois partent pendant leur première année, contre moins d'un sur trois au delà de sept ans. Le montant mensuel joue ensuite un rôle net, les clients qui partent payant en moyenne 13 % de plus, suivi des retards de paiement.

L'âge et le revenu, eux, n'ont aucun lien avec le départ (corrélation proche de zéro). C'est cohérent avec la formule de génération des données, qui ne les utilise pas. Ils ont été conservés pour respecter le contrat d'interface de l'API, mais leur retrait est une piste évidente de simplification.

Comparaison des modèles

Trois modèles ont été entraînés sur 80 % des clients, avec un découpage stratifié pour garder 43 % de churn de chaque côté. Le choix ne se fait pas sur le jeu de test mais sur le F1 moyen en validation croisée à 5 plis, calculé uniquement sur l'entraînement : le jeu de test reste ainsi un juge neutre.

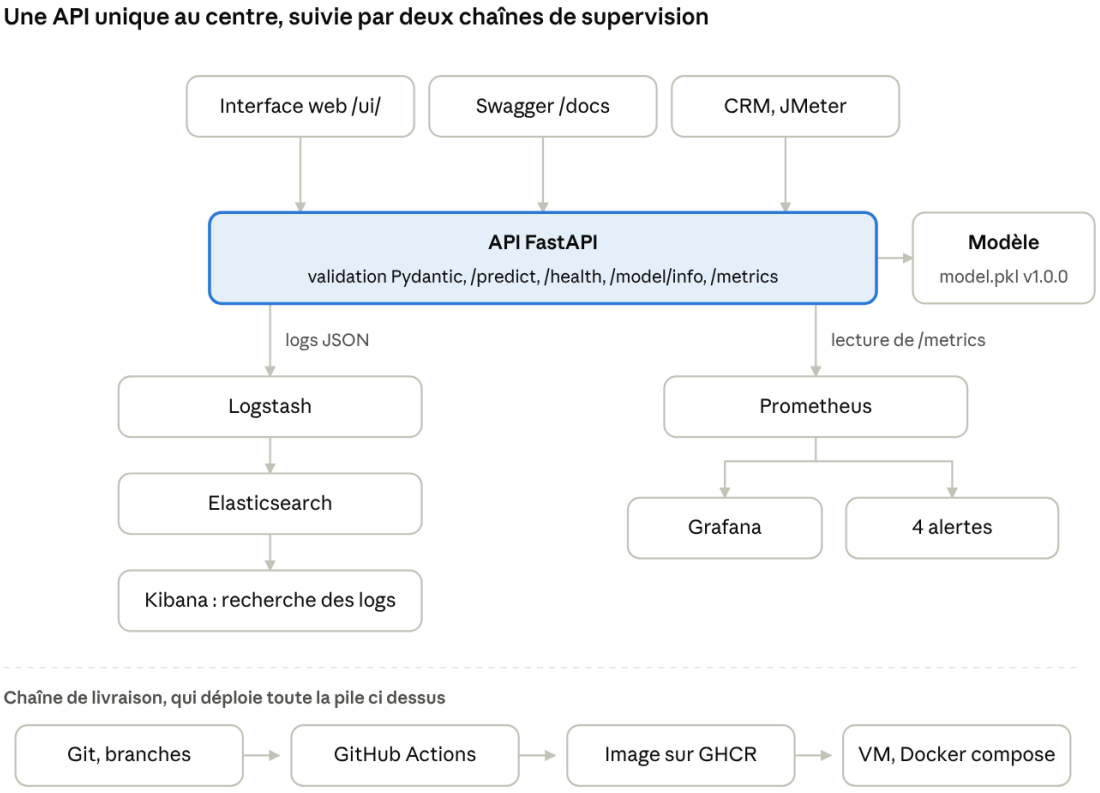
Modèle	Accuracy	Précision	Rappel	F1 test	ROC AUC	F1 validation croisée
Régression logistique (retenue)	0,625	0,582	0,453	0,510	0,636	**0,523**
Arbre de décision	0,633	0,583	0,512	0,545	0,623	0,436
Forêt aléatoire	0,635	0,618	0,395	0,482	0,639	0,478

Sur le jeu de test, les trois modèles se tiennent de près. La validation croisée les départage : la régression logistique obtient le meilleur F1 avec une variance faible, alors que l'arbre de décision, qui brille sur le test, se montre très instable d'un pli à l'autre. Ce choix est aussi logique sur le fond, puisque les données proviennent d'une formule logistique. Ses coefficients sont par ailleurs faciles à expliquer aux équipes métier.

La performance reste modeste, avec une ROC AUC de 0,64. Elle s'explique par le bruit du jeu de données : pour beaucoup de clients, la probabilité théorique de départ tourne autour de 0,5. Le modèle doit donc être vu comme un outil de priorisation des clients à contacter, pas comme une certitude individuelle.

3. Architecture de la solution

Toute la solution s'organise autour d'une API unique, qui charge le modèle et répond aux demandes de prédiction. Deux chaînes de supervision l'observent en permanence, et une chaîne de livraison automatisée la construit et la déploie.



architecture de la solution · 6 services Docker et chaîne de livraison

Les logs suivent la colonne de gauche jusqu'à Kibana, les métriques celle de droite jusqu'à Grafana et aux alertes. Chaque brique tourne dans son propre conteneur, et l'ensemble démarre en une seule commande.

4. L'API de prédiction

Le modèle est servi par une API FastAPI. Ce choix tient en trois raisons : FastAPI valide automatiquement les entrées grâce à Pydantic, génère sa documentation Swagger sans effort, et s'intègre facilement avec Prometheus. N'importe quelle application, qu'il s'agisse d'un CRM ou de l'interface d'un conseiller, peut ainsi obtenir une prédiction par un simple appel HTTP, sans rien savoir de Python ni du modèle.

Route	Méthode	Rôle
`/predict`	POST	Reçoit les six caractéristiques d'un client, renvoie CHURN ou NO_CHURN avec la probabilité

Route	Méthode	Rôle
`/health`	GET	Sonde technique : 200 si le modèle est chargé, 503 sinon
`/model/info`	GET	Nom, version, date d'entraînement et métriques du modèle servi
`/metrics`	GET	Métriques au format Prometheus
`/docs`	GET	Documentation interactive Swagger
`/ui/`	GET	Interface web de démonstration

Pour le client d'exemple de l'énoncé (35 ans, 5 mois d'ancienneté, 2 réclamations, 2 retards), l'API répond CHURN avec une probabilité de 0,599, en quelques millisecondes.

Plusieurs choix rendent cette API fiable en exploitation :

- **Validation métier des entrées.** Un âge négatif ou un champ manquant est refusé avec une erreur 422 explicite, au lieu de produire une prédiction absurde.
- **Colonnes nommées.** Les données sont transmises au modèle avec la liste de variables enregistrée à l'entraînement. Un changement d'ordre des champs ne peut donc pas fausser silencieusement le résultat.
- **Version lue, pas écrite en dur.** Le nom et la version du modèle viennent du fichier `metadata.json` produit à l'entraînement. Ils restent justes quand on change de modèle.
- **Sonde de santé honnête.** `/health` renvoie 503 tant que le modèle n'est pas chargé, ce qui permet à Docker et au répartiteur de charge de ne pas envoyer de trafic à une instance qui ne sait pas répondre.
- **Traçabilité.** Chaque requête reçoit un identifiant unique, renvoyé dans l'en tête `X-Request-ID` et repris dans les logs. On peut ainsi retrouver en quelques secondes le détail d'une prédiction contestée.

Une petite interface web, servie sur `/ui/`, complète l'ensemble : elle permet de saisir un client et d'afficher le résultat, et sert de support aux tests Selenium.

5. Stratégie de tests

Le projet compte 21 tests automatisés, organisés sur trois niveaux. Chaque niveau répond à une question différente, et c'est leur combinaison qui donne confiance avant une livraison.

Niveau	Outil	Nombre	Ce qui est vérifié
Unitaire	Pytest	5	Schéma du dataset, cohérence des métadonnées, probabilités valides, seuil de qualité du modèle, effet attendu de l'ancienneté
Intégration	Pytest et TestClient	13	Toutes les routes de l'API, prédictions déterministes, refus des valeurs invalides et des champs manquants, exposition des métriques
Bout en bout	Selenium	3	Swagger accessible, prédiction depuis l'interface

Niveau	Outil	Nombre	Ce qui est vérifié
			web, message d'erreur sur une saisie invalide

Deux tests méritent d'être soulignés. Le premier est un **seuil de qualité** : si un réentraînement produit un modèle dont la ROC AUC passe sous 0,60, la CI échoue et le modèle ne peut pas être déployé. Le second est un **test de bon sens métier** : à profil égal, un client très ancien doit avoir un risque plus faible qu'un client récent. Il détecterait un modèle qui aurait appris l'inverse de la réalité.

Les tests Selenium pilotent un vrai navigateur Chrome, en mode sans affichage pour pouvoir tourner sur un serveur ou dans la CI. Ils saisissent un client, cliquent sur Prédire et lisent le résultat affiché, exactement comme le ferait un utilisateur. Plus lents et plus fragiles que les autres, ils sont isolés par un marqueur `e2e` : `pytest` lance les 18 tests rapides, `pytest -m e2e` les 3 tests navigateur.

6. Conteneurisation

Docker résout le fameux « ça marche sur ma machine » : l'application, ses dépendances et le modèle sont empaquetés dans une image identique partout, du poste de développement à la machine virtuelle de production.

Le `Dockerfile` part d'une image `python:3.12-slim` et applique les bonnes pratiques habituelles. Les dépendances sont installées dans une couche séparée, mise en cache tant que `requirements.txt` ne change pas. Un fichier `.dockerignore` évite d'embarquer l'environnement virtuel, l'historique Git ou les logs. L'application tourne sous un utilisateur sans privilèges, et un `HEALTHCHECK` interroge `/health` toutes les 15 secondes.

Un choix structurant mérite d'être expliqué : **le modèle est entraîné pendant la construction de l'image**. Un modèle sérialisé avec joblib ne se recharge de façon fiable qu'avec les mêmes versions de Python et de scikit learn. En l'entraînant dans l'image, on élimine ce risque, et l'image ne peut plus jamais être construite sans modèle. La graine fixe garantit que le résultat reste identique d'une construction à l'autre.

Le fichier `docker-compose.yml` orchestre ensuite toute la pile en une seule commande, `docker compose up -d --build` :

Service	Image	Rôle
api	ml-churn-api:1.0.0	Prédiction, logs JSON, métriques
elasticsearch	Elasticsearch 8.15.3	Indexation des logs
logstash	Logstash 8.15.3	Collecte et transformation des logs
kibana	Kibana 8.15.3	Recherche et visualisation des logs
prometheus	Prometheus 2.55.1	Collecte des métriques et alertes
grafana	Grafana 11.3.0	Tableaux de bord

Les six services partagent un réseau commun, des volumes persistants et des sondes de santé qui règlent l'ordre de démarrage. La datasource et le dashboard Grafana sont provisionnés automatiquement : rien n'est à configurer à la main après le lancement.

7. Intégration continue

Chaque envoi de code sur GitHub déclenche automatiquement le pipeline « CI ML Churn ». Il s'organise en trois jobs :

1. **Tests unitaires et API.** Installation de l'environnement, génération des données, entraînement du modèle, exécution des 18 tests Pytest, puis archivage du modèle et de ses métriques comme artefact du build.
2. **Tests E2E Selenium.** Lancés seulement si le premier job réussit, ils exécutent le scénario utilisateur complet dans Chrome sans affichage.
3. **Build Docker.** Construction de l'image, test de fumée du conteneur (appel de `/health` puis de `/predict`), et publication sur le registre GitHub Container Registry lorsque la branche est `main`.

Le premier passage complet a réussi les trois jobs en 1 minute 39 secondes. La conduite du dépôt suit le schéma classique : les développements se font sur des branches de fonctionnalité, fusionnées dans `develop`, et `main` n'évolue que par Pull Request une fois la CI au vert.

L'intérêt est double. Une erreur est détectée quelques minutes après avoir été introduite, bien avant la production. Et la livraison devient répétable : l'image publiée par la CI est exactement celle qui sera déployée, il suffit de la récupérer avec `docker pull` sur la machine cible.

Un second workflow, déclenché à la demande, démarre toute la pile sur un serveur GitHub, lance les tirs de charge et prend les captures d'écran des outils. Les preuves présentées dans ce rapport sont donc reproductibles par n'importe qui, en un clic.

8. Observabilité

Une fois en production, un modèle peut tomber en panne de deux façons : comme n'importe quel service (il ne répond plus, il ralentit, il renvoie des erreurs), ou de façon silencieuse, en continuant de répondre alors que ses prédictions ne veulent plus rien dire. L'observabilité mise en place couvre les deux cas, en combinant deux outils complémentaires : les logs pour enquêter, les métriques pour surveiller et alerter.

Les logs avec ELK

L'API écrit un log JSON structuré pour chaque requête. On y trouve l'identifiant de la requête, l'adresse du client, le modèle et sa version, les données reçues, la prédiction, la probabilité et le temps de réponse, ainsi que la trace complète en cas d'erreur. Logstash lit ces fichiers depuis un volume partagé, les indexe dans Elasticsearch, et Kibana permet de les explorer.

Pendant les tests de charge, Kibana a reçu 16 001 logs de prédiction, et la recherche `level : "ERROR"` n'a retourné aucun résultat. En exploitation, des requêtes simples permettent de répondre aux questions concrètes : quelles prédictions ont dépassé 50 ms, quels clients ont été classés CHURN avec une probabilité supérieure à 0,8, ou que s'est il passé pour une requête précise.

Les métriques avec Prometheus et Grafana

Prometheus interroge la route `/metrics` de l'API à intervalle régulier. Aux métriques HTTP classiques (nombre de requêtes, codes de réponse, latence) s'ajoutent des métriques propres au modèle : nombre de prédictions par résultat, erreurs de prédiction, durée d'inférence, distribution des probabilités et état de chargement du modèle.

Le dashboard Grafana « ML Churn API : supervision » rassemble ces indicateurs sur un seul écran : volume de requêtes, taux d'erreur, latence aux 50e, 95e et 99e percentiles, répartition des prédictions, part de CHURN et distribution des probabilités.

Les alertes

Quatre règles d'alerte, validées avec l'outil `promtool`, complètent le dispositif :

Alerte	Déclenchement	Gravité
ApiDown	L'API ne répond plus pendant 1 minute	Critique
HighErrorRate	Plus de 5 % de réponses 5xx pendant 5 minutes	Avertissement
HighLatencyP95	95e percentile de `/predict` au dessus de 500 ms pendant 5 minutes	Avertissement
PredictionDrift	Plus de 80 % de CHURN prédits sur une heure, pendant 30 minutes	Avertissement

La dernière est la plus intéressante du point de vue MLOps. Le taux de churn observé à l'entraînement est de 43 % : si le modèle se met soudain à classer presque tous les clients comme partants, c'est le signe que les données reçues ont changé de nature, ou que quelque chose s'est cassé en amont. C'est exactement le type de panne silencieuse que la supervision technique seule ne verrait pas.

9. Tests de performance

L'API tient 1 000 utilisateurs simultanés sans une seule erreur, mais son débit plafonne vers 220 requêtes par seconde : au delà, les utilisateurs supplémentaires se traduisent uniquement en attente.

Trois tirs ont été menés avec JMeter en mode non graphique, sur un serveur de 4 processeurs virtuels. Chaque utilisateur virtuel envoie 10 requêtes `/predict`, avec une montée en charge sur 10 secondes et une assertion qui vérifie le contenu de chaque réponse.

Utilisateurs	Requêtes	Erreurs	Temps moyen	Médiane	90e percentile	95e percentile	99e percentile	Débit
100	1 000	0 %	6 ms	4 ms	9 ms	13 ms	19 ms	102 req/s
500	5 000	0 %	1 361 ms	1 475 ms	2 012 ms	2 058 ms	2 097 ms	216 req/s

Utilisateurs	Requêtes	Erreurs	Temps moyen	Médiane	90e percentile	95e percentile	99e percentile	Débit
1 000	10 000	0 %	3 564 ms	4 045 ms	4 438 ms	4 518 ms	4 652 ms	222 req/s

À 100 utilisateurs, la charge correspond à ce que l'API absorbe sans effort : 95 % des réponses arrivent en moins de 13 ms. Le débit de 102 requêtes par seconde est simplement celui que JMeter génère, l'API n'est pas encore sollicitée à son maximum.

À 500 puis 1 000 utilisateurs, on passe en test de stress. Le débit ne progresse presque plus (216 puis 222 requêtes par seconde) : l'API a atteint sa capacité. Les requêtes en trop attendent leur tour, et le temps de réponse grimpe en proportion, jusqu'à 4,5 secondes au 95e percentile. Le service se dégrade donc proprement, par l'attente, sans jamais casser.

Ce plafond s'explique par l'architecture choisie : un seul processus uvicorn, qui ne profite que d'un cœur de processeur. Le modèle lui même n'est pas en cause, puisqu'une inférence prend quelques millisecondes. Pour monter en charge, il suffirait de lancer plusieurs workers ou plusieurs conteneurs derrière un répartiteur.

Le suivi en parallèle dans Prometheus a d'ailleurs révélé un piège instructif. Avec les réglages par défaut, le 95e percentile affiché restait bloqué à 1 seconde pendant le tir à 500 utilisateurs, alors que les requêtes en duraient plus de 2. L'histogramme de latence s'arrêtait tout simplement à 1 seconde. Après extension des intervalles jusqu'à 30 secondes, le p95 côté serveur est mesuré à sa vraie valeur, 2,44 secondes. Un tableau de bord n'est fiable que si l'on sait comment ses chiffres sont calculés.

10. Cycle de vie du modèle

Industrialiser un modèle ne s'arrête pas à son premier déploiement : il faudra le réentraîner, le remplacer, parfois revenir en arrière. Le projet pose les bases de cette gestion.

Versionnement. Chaque entraînement produit le modèle courant `model.pkl`, une copie versionnée `model_v1.0.0.pkl` et un fichier `metadata.json`. Ce dernier enregistre la version, la date, les variables, les métriques des trois modèles comparés et les versions des librairies. La version servie est exposée par `/model/info`, renvoyée dans chaque réponse et portée par chaque log. On sait donc toujours quel modèle a produit quelle prédiction.

Sélection automatique. Le modèle retenu n'est pas imposé à l'avance : le script d'entraînement choisit celui qui obtient le meilleur F1 en validation croisée. Si de nouvelles données rendaient la forêt aléatoire meilleure, elle serait retenue sans toucher au code. Le choix peut aussi être forcé par une variable d'environnement.

Garde fou. Un nouveau modèle moins bon ne doit pas partir en production. Le seuil de qualité vérifié par la CI bloque tout modèle dont la ROC AUC passe sous 0,60.

Retour arrière. Chaque version correspond à une image Docker étiquetée. Revenir à la version précédente consiste à relancer l'ancienne image, ce qui prend quelques secondes puisque rien n'est reconstruit.

Détection de la dégradation. L'alerte de dérive surveille la part de CHURN prédite. Pour aller plus loin, il faudra comparer les prédictions aux départs réels une fois ceux ci connus, et déclencher un réentraînement quand la performance baisse.

11. Difficultés rencontrées

Suivre l'énoncé à la lettre ne suffisait pas : 31 points ont dû être corrigés, et chacun a été vérifié par exécution. Le détail complet figure dans le fichier `docs/CORRECTIONS.md` du dépôt.

Gravité	Nombre	Conséquence sans correction
Bloquant	9	Le pipeline ne fonctionne pas : erreur, test toujours en échec, service injoignable
Important	14	Le pipeline tourne, mais le résultat est faux, non reproductible ou peu industriel
Amélioration	8	Bonnes pratiques MLOps attendues en soutenance

Quelques exemples illustrent la nature de ces difficultés.

Les tests ne se lançaient pas. Pytest ne trouvait pas le module de l'API, faute de configuration du chemin Python. Un fichier `pytest.ini` et quelques fichiers `__init__.py` ont réglé le problème.

Des versions incompatibles. L'énoncé utilisait Python 3.11 dans l'image et la CI, alors que les dépendances étaient figées sur un poste en Python 3.12. Certaines versions n'existant pas pour 3.11, la construction de l'image échouait. Tout le projet a été aligné sur Python 3.12.

Une supervision qui ne voyait rien. Grafana était lancé en dehors du réseau Docker et ne pouvait pas joindre Prometheus, qui lui même n'était démarré nulle part. Aucune configuration ELK n'était fournie. La pile a été entièrement reconstruite dans un seul fichier compose.

Un résultat attendu irréaliste. L'énoncé annonçait une probabilité de 0,87 pour le client d'exemple. Le calcul à partir de la formule de génération donne environ 0,60, ce que le modèle retrouve bien. C'est la valeur réelle qui a été documentée.

Un plan de charge inopérant. Sans l'en tête `Content-Type: application/json`, toutes les requêtes JMeter recevaient une erreur 422, et sans assertion ces erreurs n'étaient même pas comptées. Le plan a été corrigé et rendu paramétrable.

Ces écarts ont finalement été très formateurs. Ils montrent qu'en MLOps, l'essentiel du travail se joue dans les détails d'intégration entre les outils, bien plus que dans le modèle lui même.

12. Limites et perspectives

Le projet remplit ses objectifs pédagogiques, mais plusieurs points devraient être traités avant une mise en production réelle.

Côté modèle, la performance reste limitée par des données synthétiques et bruitées. Sur de vraies données d'opérateur, on pourrait enrichir les variables (usage, type d'offre, historique des contacts avec

le service client), ajuster le seuil de décision selon le coût d'une action de rétention, et retirer les variables inutiles comme l'âge et le revenu.

Côté performance, une seule instance uvicorn plafonne à 220 requêtes par seconde. Plusieurs workers ou plusieurs conteneurs derrière un répartiteur lèveraient cette limite. Il faudrait alors activer le mode multiprocessus de la librairie Prometheus, car les métriques sont aujourd'hui tenues par processus.

Côté sécurité, l'API n'a pas d'authentification et la sécurité d'Elasticsearch est désactivée pour les besoins du TP. Une clé d'API ou OAuth2, le chiffrement TLS et des mots de passe gérés comme des secrets seraient indispensables.

Côté MLOps, le versionnement repose sur des fichiers. Un registre de modèles comme MLflow, le versionnement des données avec DVC, un déploiement progressif de type canary et un réentraînement déclenché par la dérive constitueraient les prochaines étapes naturelles.

13. Conclusion

Ce projet a permis de parcourir toute la chaîne qui mène d'un modèle de Data Science à un service exploitable. Le modèle de prédiction du churn est aujourd'hui accessible par une API documentée, couvert par 21 tests, livré automatiquement dans une image Docker et surveillé en continu par des logs, des métriques et des alertes. Les tests de charge ont confirmé sa robustesse et mis en évidence le levier à actionner pour monter en charge.

La leçon principale est que le modèle n'est qu'une petite partie du travail. L'essentiel de l'effort a porté sur ce qui l'entoure : la reproductibilité, la validation des entrées, l'automatisation, la traçabilité et la capacité à voir ce qui se passe une fois le service lancé. C'est cet environnement qui transforme une expérience en un outil sur lequel une entreprise peut s'appuyer.

Transposée dans un contexte opérateur, cette démarche permettrait aux équipes de relation client de recevoir chaque jour une liste priorisée des clients à risque, avec la garantie que le modèle qui la produit est testé, versionné et surveillé.